

Architecture Report

Activite 3 – Layered Swing Application with MVP

HIMEUR Idris – DIAR Adem
2CS SIQ1 2025/2026

April 18, 2026

Contents

1	Introduction	2
2	Development View	2
3	Execution View	3
4	Deployment View	3
5	Figure References in Text	4
6	Conclusion	4

1. Introduction

This report documents the implemented architecture of `activite3`. The system is a single Java Swing desktop application, structured with explicit layers and interface-based boundaries. The source root is `activite3/src/calculator/`, and the entry point is `calculator.app.CalculatorApplication`. The build scripts compile `src/` to `out/` and run the application with local classpath dependencies `out` and `lib/interpreter.jar`.

The architectural objective is to separate concerns while preserving the calculator behavior. The presentation layer follows MVP with `CalculatorView`, `SwingCalculatorView`, and `CalculatorPresenter`. The service layer orchestrates use cases through `CalculatorService` and `CalculatorApplicationService`. External computation is isolated behind `ComputationGateway` and implemented by `InterpreterComputationGateway`. Trace persistence is isolated behind `TraceRepository` and implemented by `FileTraceRepository`, which writes a fixed-width UTF-8 table to `data/track.trc`.

2. Development View

The development view is organized by packages that correspond to technical responsibilities: `app`, `presentation.view`, `presentation.presenter`, `service`, `adapter`, `persistence`, and `domain`. The `app` layer wires concrete implementations in `CalculatorApplication.main(...)`.

MVP is explicit in presentation. `CalculatorView` defines the UI contract, and `SwingCalculatorView` provides the Swing implementation with labels and controls `Nombre 1`, `Nombre 2`, `Somme`, `Produit`, `Factoriel`, `Nombre Aleatoire`, `Trace`, and `Quitter`. `CalculatorPresenter` binds actions, validates input, delegates business operations to `CalculatorService`, and updates the view.

Interface-based boundaries reduce direct coupling: `presenter` depends on `CalculatorView` and `CalculatorService`; `CalculatorApplicationService` depends on `ComputationGateway` and `TraceRepository`. The adapter `InterpreterComputationGateway` implements `ComputationGateway` and is the integration point with `interpreter.Evaluator` for `add`, `multiply`, and `factorial` using postfix expressions. The `random()` method is local and uses `java.util.Random` with `nextInt(10000)`.

The domain model is intentionally compact: `OperationType` captures operation categories and display names, and `TraceEntry` captures operation operands and result values used by service and persistence.

library. The trace is persisted in the local file `data/track.trc`.

There is no server node, no distributed runtime, no remote API call, and no database service. The architecture is therefore suitable for a standalone academic desktop application where execution, dependency loading, and persistence all occur on the same machine.

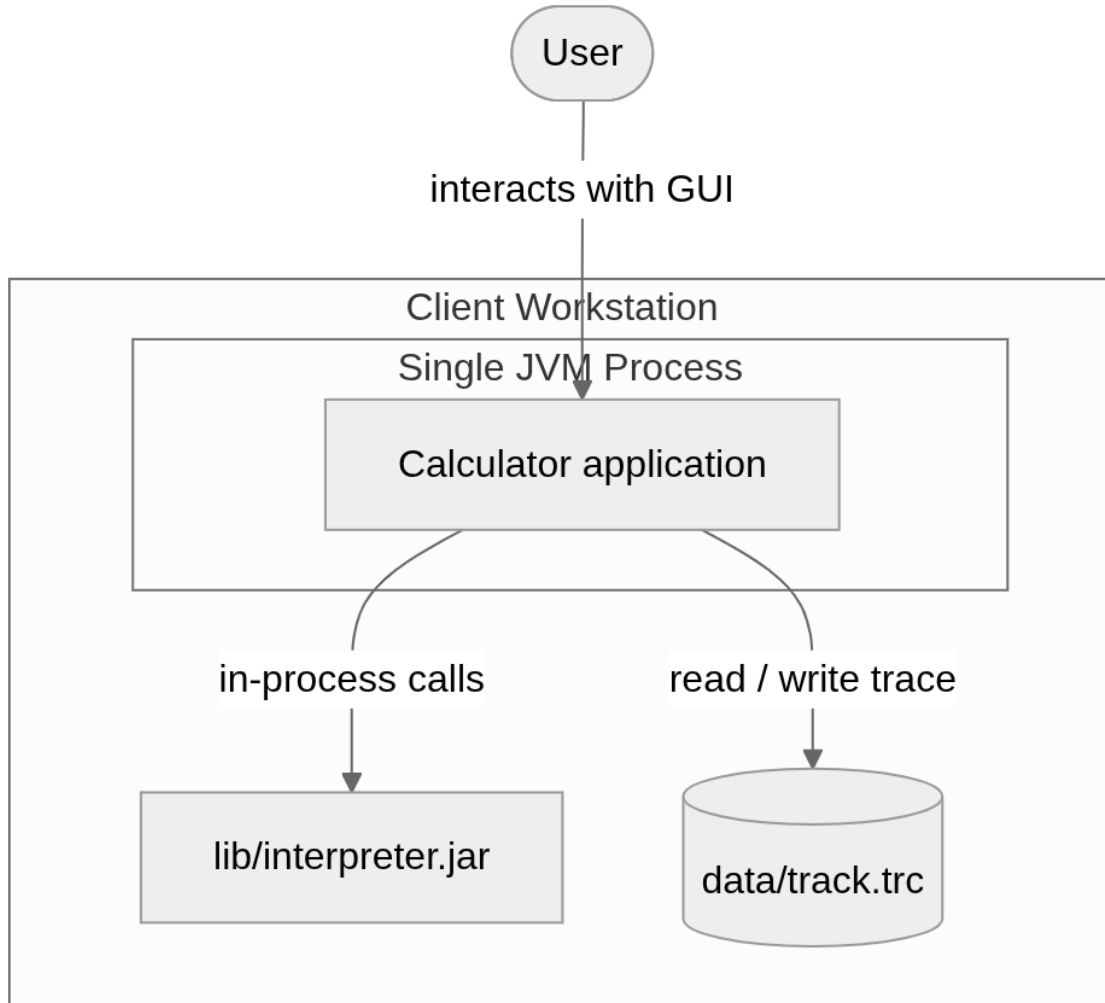


Figure 3: Deployment view: single-machine desktop runtime with local JAR dependency and local trace file persistence.

5. Figure References in Text

Figure 1 summarizes static structure and interface boundaries. Figure 2 shows the runtime control path for computation and trace loading. Figure 3 presents the local, single-node deployment model.

6. Conclusion

The implemented `activite3` architecture is a layered Swing desktop design with MVP in the presentation layer and clear interface-based integration boundaries. The service layer centralizes use-case orchestration, the adapter isolates interpreter-specific computation logic, and persistence remains independent from Swing while storing trace data in a readable fixed-width text

table. This structure improves clarity, testability at boundary level, and maintainability while preserving the intended functional behavior.